# UE23CS352B - Object Oriented Analysis & Design

## Mini Project Report

## Online Appointment Scheduling System

***Submitted by:***

***Team Members***


| NAME | SRN |
|---|---|
| Rithvik Matta | PES2UG23CS485 |
| Rishil Abhijit Jalisatgi | PES2UG23CS482 |
| Retesh G S | PES2UG23CS475 |
| Rishi A Sheth | PES2UG23CS479 |

Sem 6 Section H

**FACULTY NAME : Dr. Vanitha**

**January - May 2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
FACULTY OF ENGINEERING
**PES UNIVERSITY**
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

# 1. Problem Statement

The Online Appointment Scheduling System is a web-based healthcare management platform designed to digitize and streamline the process of booking, managing, and tracking medical appointments. The system eliminates manual scheduling, reduces waiting times, and provides a centralized platform for patients, doctors, and administrators to interact efficiently.

The system supports three distinct user roles: Patients, who register and book appointments with available doctors; Doctors, who manage their schedules and handle appointment requests; and Administrators, who oversee the entire system including user management, reporting, and analytics.

Before an appointment is confirmed, the system validates slot availability and doctor existence. Patients can view their booking history, reschedule appointments, and make payments. Doctors can approve, reject, or mark appointments as complete. Administrators have access to system-wide reports, revenue summaries, and full user management capabilities. The system also implements a real-time in-app notification mechanism to keep all parties informed of status changes.

## Key Features

**Major Features:**

- Appointment Booking & Management – Patients search for doctors, view available time slots, and book appointments with reason of visit
- Doctor Schedule Management – Doctors add/remove availability slots and approve or reject patient appointment requests
- Payment Processing – Patients can pay for approved appointments via Card, Cash, or UPI (Strategy Pattern)
- Admin Dashboard & Reports – Administrators view system-wide statistics, manage users, generate revenue reports

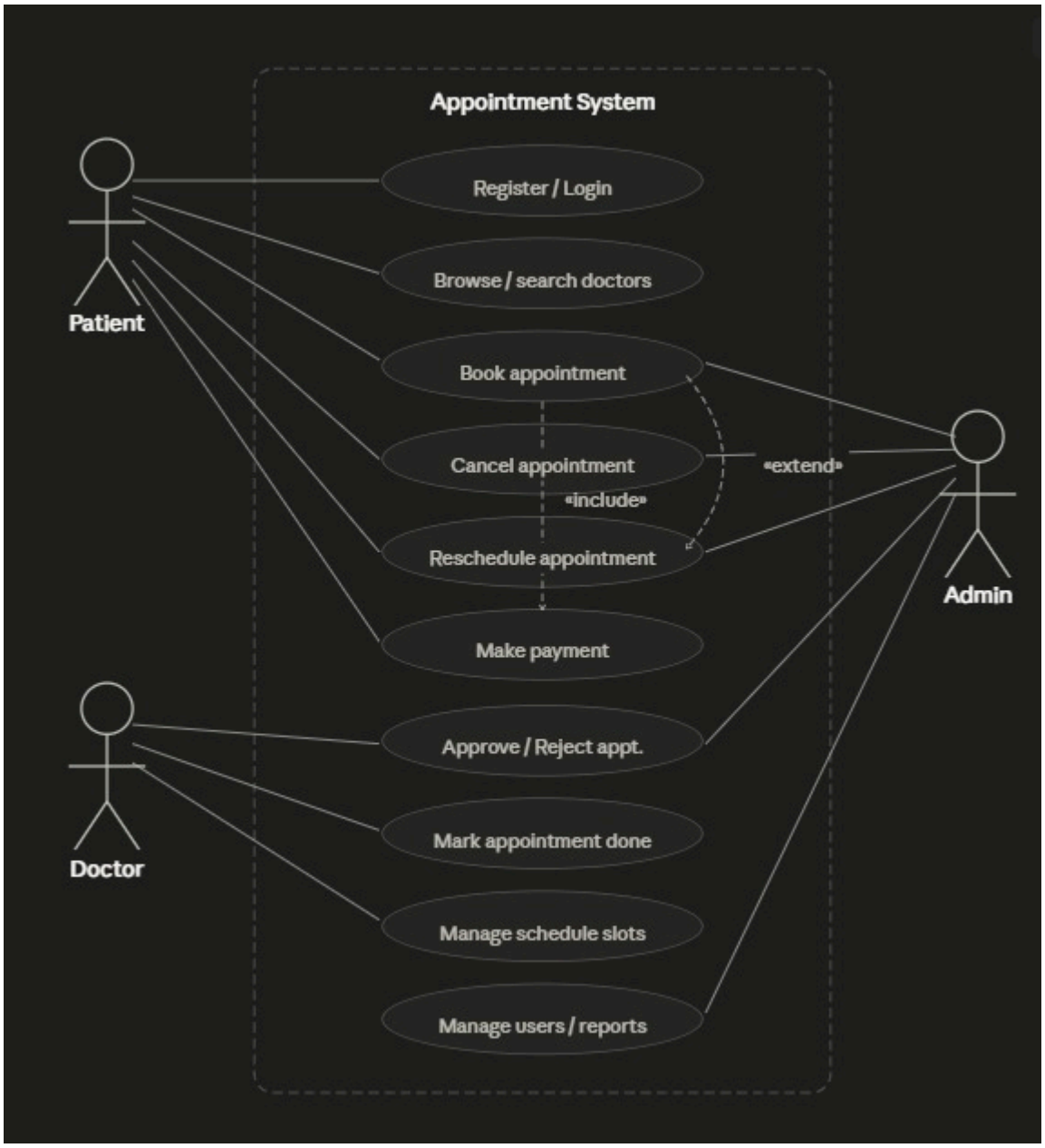
**Minor Features:**

- In-App Notification System – Real-time notifications for appointment status changes for both doctors and patients
- Appointment Rescheduling – Patients can reschedule pending appointments to a different available slot
- Appointment Cancellation – Patients can cancel appointments, which automatically frees up the doctor's slot
- User Profile Management – Users can update their profile name and phone number

# 2. UML Models

## 2.1 Use Case Diagram

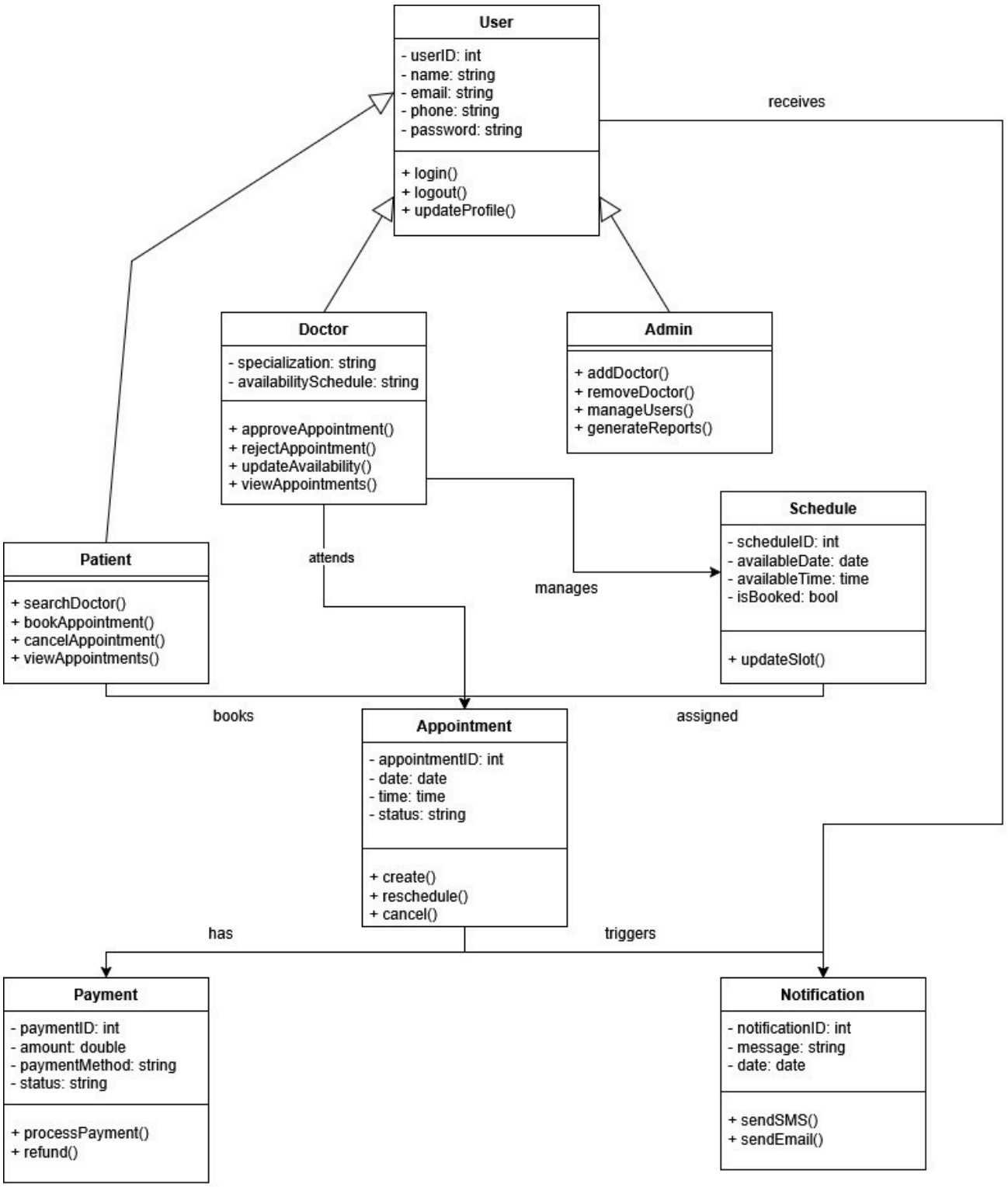
The Use Case Diagram below illustrates the interactions between the three actors (Patient, Doctor, Administrator) and the system's functional use cases. Each actor owns a distinct set of operations reflecting their role in the appointment lifecycle.




Primary use cases include: Register/Login, Book Appointment, Manage Schedule, Approve/Reject Appointment, Process Payment, View Notifications, and Generate Reports.

## 2.2 Class Diagram

The Class Diagram depicts the core domain entities and their relationships. The main classes are User (abstract superclass), Patient, Doctor, Admin, Appointment, Schedule, Payment, and Notification. Inheritance is used for user types, while associations model the lifecycle of an appointment.

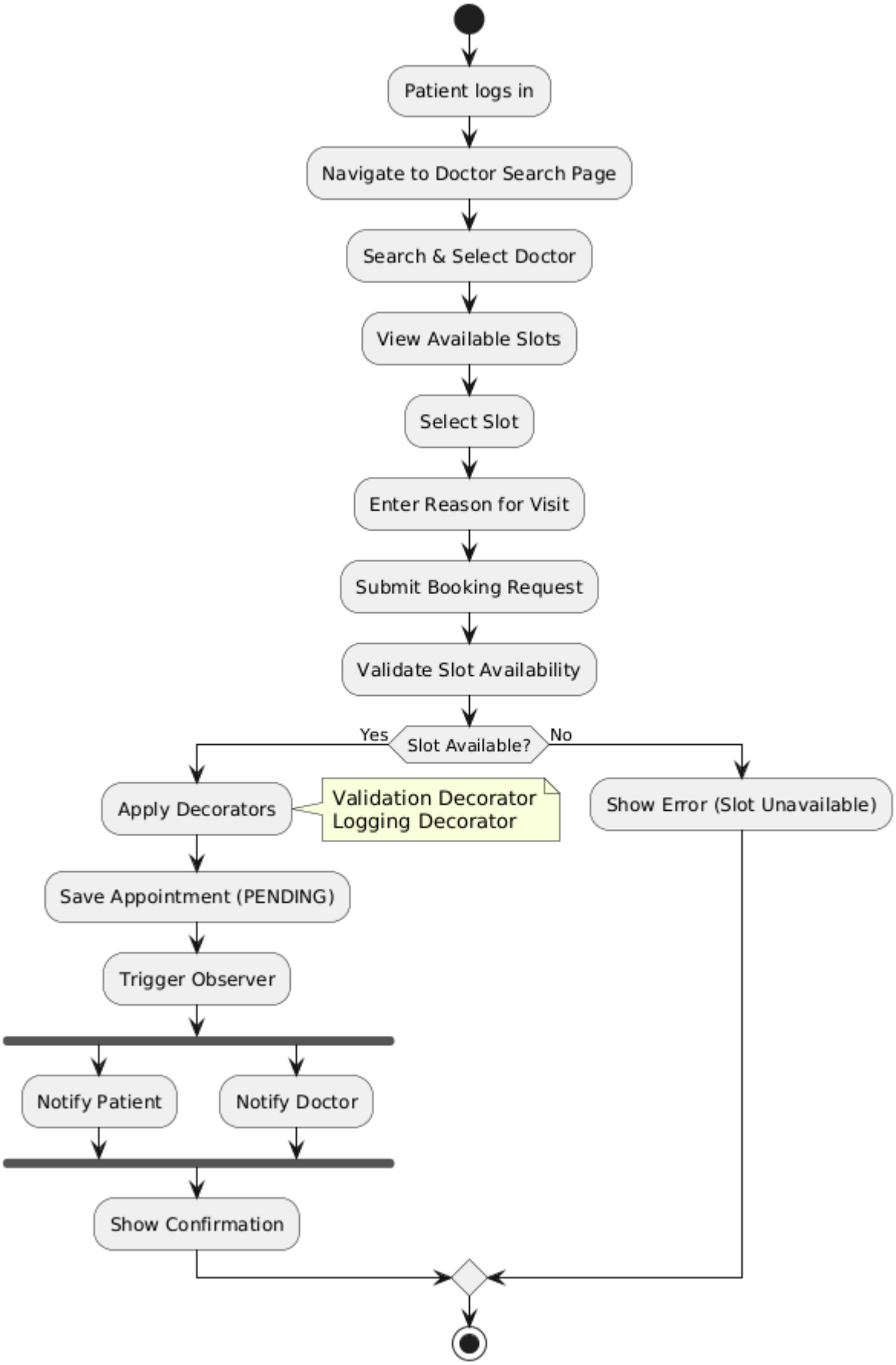


| Class | Responsibility | Key Attributes |
|---|---|---|
| User | Abstract base entity for all users | userId, name, email, phone, password, role |
| Patient | Book/cancel/reschedule appointments | Inherits from User |
| Doctor | Manage schedule, approve appointments | specialization, qualification, hospital, consultation fee |
| Admin | User management, system oversight | department |
| Appointment | Core booking entity | appointmentId, date, time, status, reason |
| Schedule | Available time slots for doctors | availableDate, availableTime, isBooked |
| Payment | Payment for an appointment | amount, method, status, paidAt |
| Notification | Status update messages | message, type, isRead, createdAt |

## 2.3 Activity Diagrams

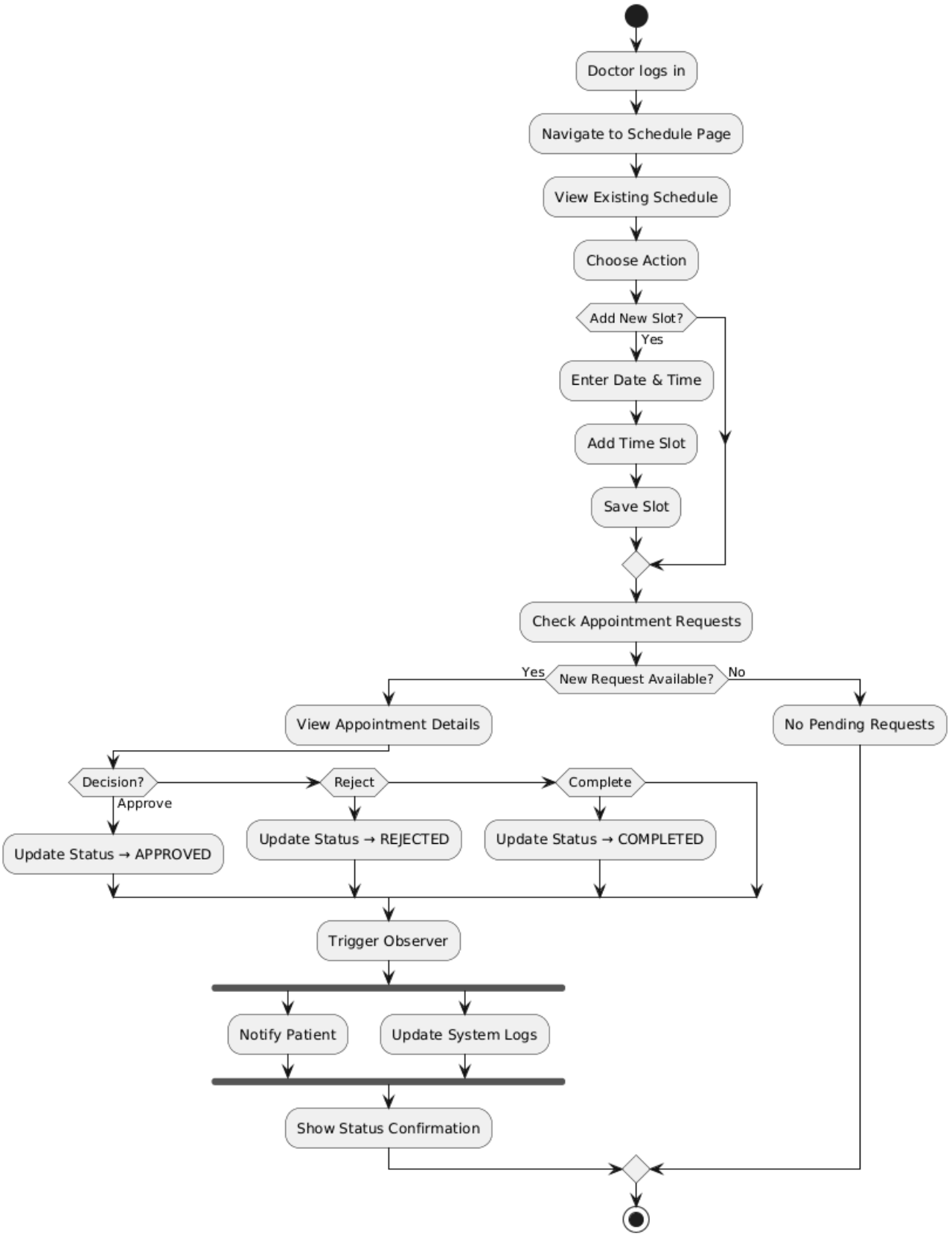
### 2.3.1 Book Appointment (Patient)

The patient logs in, navigates to the doctor search page, selects a doctor, views available slots, enters a reason for visit, and submits the booking. The system validates the slot availability using the Decorator Pattern (Validation + Logging decorators), saves the appointment with PENDING status, and triggers the Observer to notify both patient and doctor.


Patient logs in
Navigate to Doctor Search Page
Search & Select Doctor
View Available Slots
Select Slot
Enter Reason for Visit
Submit Booking Request
Validate Slot Availability
Slot Available?
Yes
No
Apply Decorators
Validation Decorator
Logging Decorator
Show Error (Slot Unavailable)
Save Appointment (PENDING)
Trigger Observer
Notify Patient
Notify Doctor
Show Confirmation

### 2.3.2 Doctor Manages Schedule & Approves Appointments

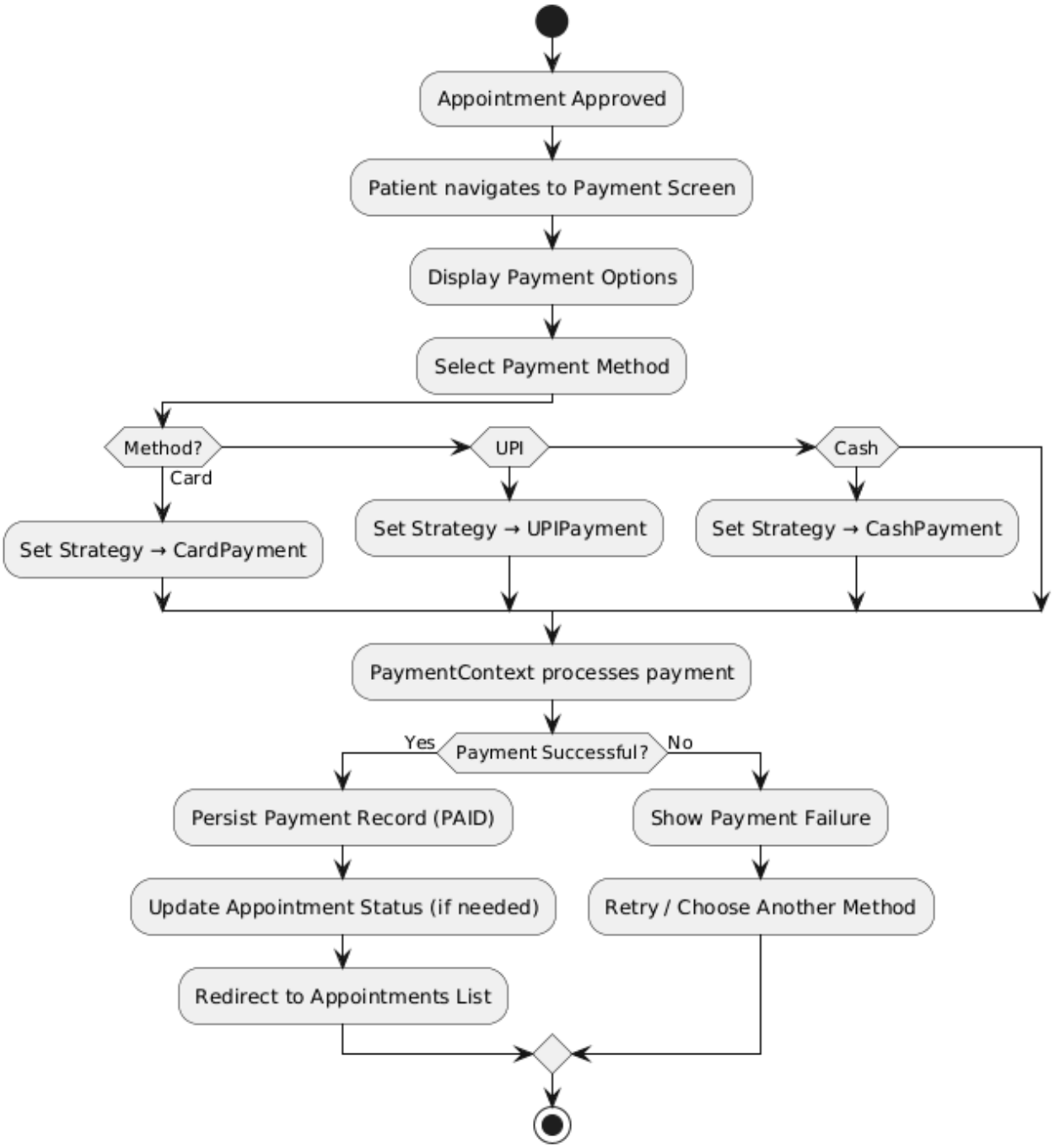
The doctor logs in and navigates to their schedule page. They add new time slots by specifying date and time. When a new appointment request appears, the doctor can approve it (status → APPROVED), reject it (status → REJECTED), or mark a completed appointment as COMPLETED. Each status change triggers the Observer notification system.




## 2.3.3 Payment Processing

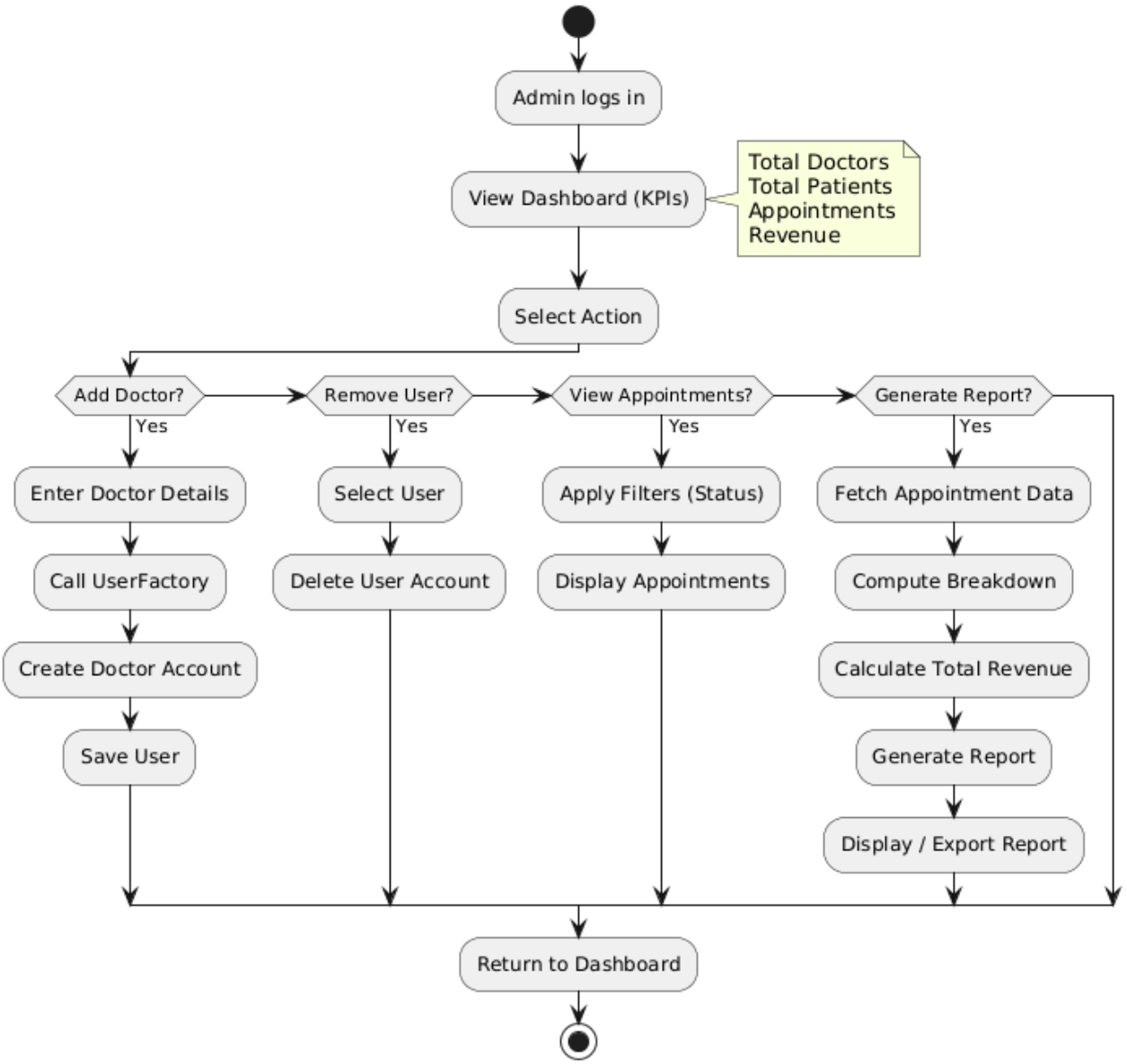
After an appointment is approved, the patient navigates to the payment screen. They select a payment method (Card, UPI, or Cash). The PaymentContext (Strategy Pattern) selects the

appropriate PaymentStrategy at runtime and processes the payment. On success, a Payment record is persisted with PAID status and the patient is redirected to their appointments list.




### 2.3.4 Admin User Management & Reporting

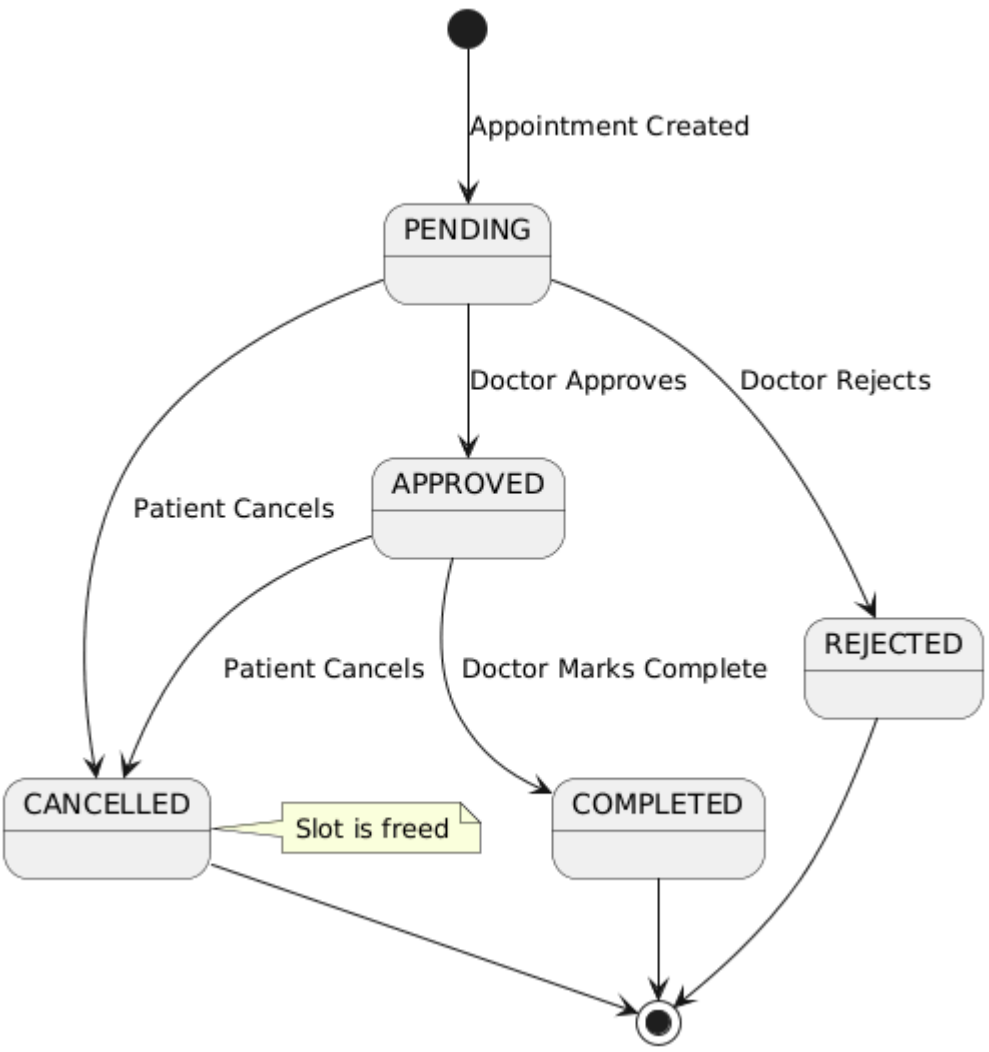
The administrator logs in to the dashboard showing system KPIs (total doctors, patients, appointments, revenue). The admin can add new doctor accounts (delegating to UserFactory for creation), remove existing users, view all appointments filtered by status, and generate a comprehensive report showing appointment breakdown and total revenue.




## 2.4 State Diagrams

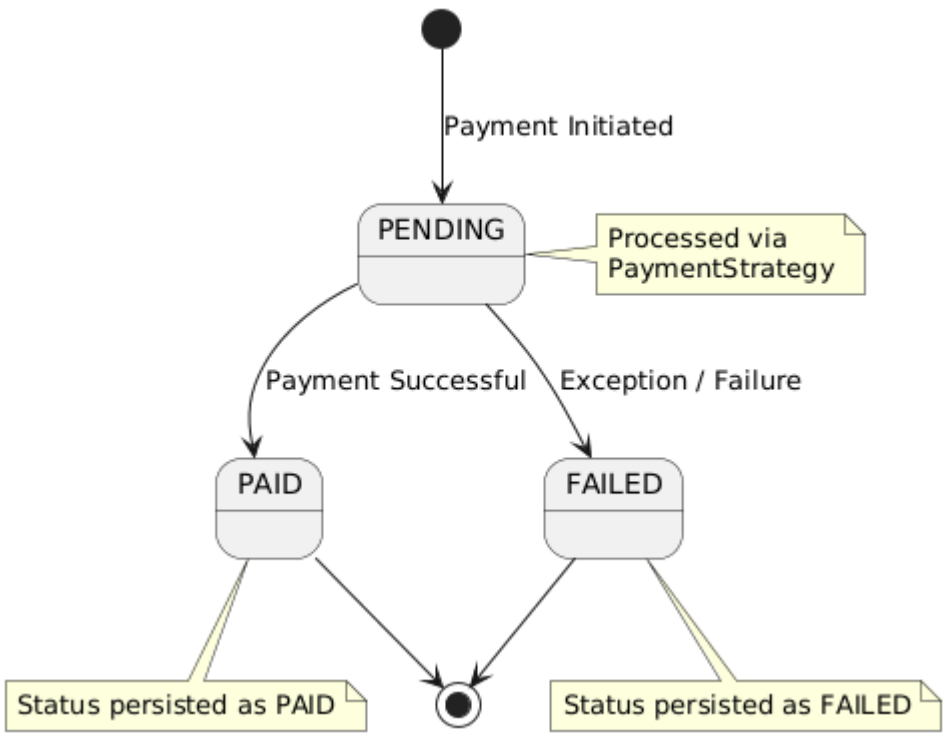
### 2.4.1 Appointment State Diagram

An appointment transitions through the following states: PENDING (initial state when booked) → APPROVED (doctor approves) or REJECTED (doctor rejects) → COMPLETED (doctor marks complete) or CANCELLED (patient cancels from PENDING or APPROVED). The Schedule slot is freed when an appointment is cancelled.



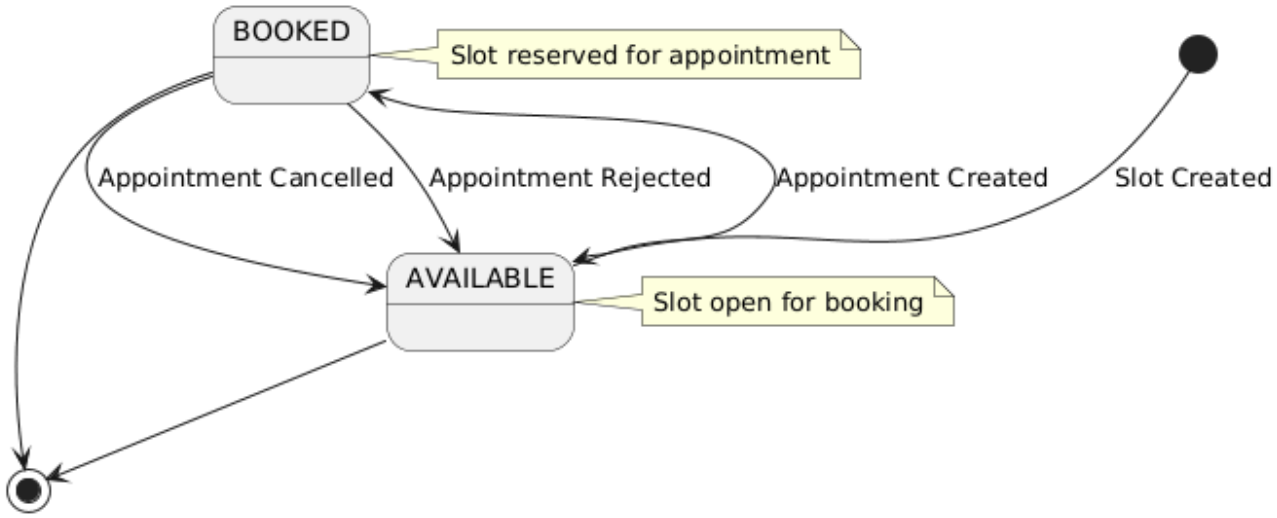

### 2.4.2 Payment State Diagram

A Payment starts in PENDING state, transitions to PAID upon successful processing, or to FAILED if the payment strategy throws an exception. The state is persisted in the Payment entity with a PaymentStatus enum.



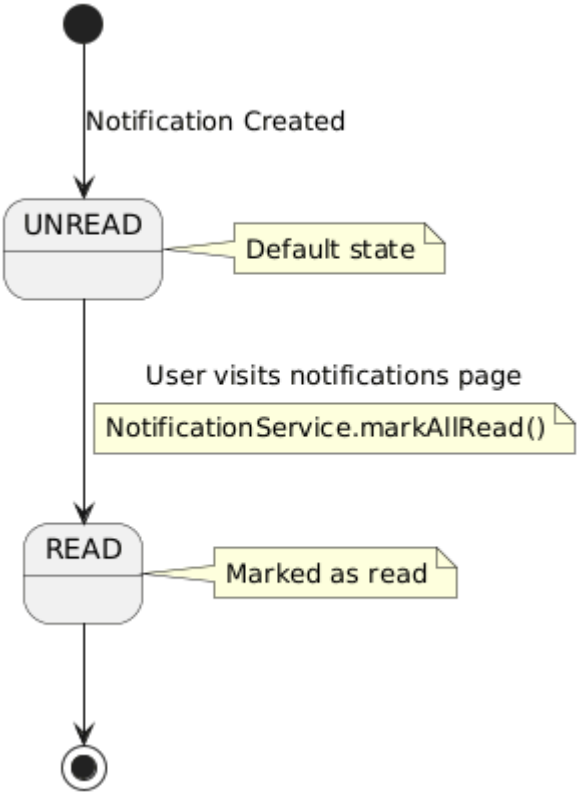

### 2.4.3 Schedule Slot State Diagram

A Schedule slot is either AVAILABLE or BOOKED. It transitions from AVAILABLE → BOOKED when an appointment is created, and from BOOKED → AVAILABLE when the appointment is cancelled or rejected.




### 2.4.4 Notification State Diagram

A Notification is created in UNREAD state. When the user visits their notifications page, all notifications are marked READ. The NotificationService.markAllRead() method handles this transition.


Notification Created
UNREAD
Default state
User visits notifications page
NotificationService.markAllRead()
READ
Marked as read

# 3. Architecture Pattern, Design Principles & Patterns

## 3.1 MVC Architecture

**MVC Architecture Used: YES**

The project strictly follows the Model-View-Controller (MVC) architectural pattern as enforced by the Spring MVC framework:

| Layer | Technology | Role in this Project |
|---|---|---|
| Model | JPA Entities + Service Layer | Domain classes (Appointment, Doctor, Patient, etc.) and business logic in Service implementations |
| View | Thymeleaf Templates | HTML templates in src/main/resources/templates/ rendered server-side for patient, doctor, and admin dashboards |
| Controller | Spring @Controller classes | PatientController, DoctorController, AdminController, AuthController handle HTTP requests, invoke services, and return view names |

The separation is enforced by Spring's dependency injection: Controllers never directly access repositories; they only interact with the Service layer. Services encapsulate all business logic and interact with JPA Repositories. Views only receive model attributes via the Spring Model object and never contain business logic.

## 3.2 Design Principles

### 3.2.1 Single Responsibility Principle (SRP)

Applied throughout the codebase: each class has exactly one reason to change. For example, AppointmentServiceImpl is solely responsible for appointment business logic; PatientController handles only HTTP request routing for patient-facing endpoints; PaymentServiceImpl handles only payment processing. The Notification creation responsibility is delegated entirely to InAppNotifier, not mixed into AppointmentServiceImpl. This clean separation ensures changes to one feature do not inadvertently affect another.

### 3.2.2 Open/Closed Principle (OCP)

The system is open for extension but closed for modification. The PaymentStrategy interface allows new payment methods to be added (e.g., NetBankingPaymentStrategy) without modifying the existing PaymentContext or any other class. Similarly, the AppointmentObserver interface allows new notification channels (e.g., EmailNotifier, SMSNotifier) to be added as additional Spring beans

without modifying AppointmentServiceImpl. The UserFactory can be extended with new user roles without changing existing factory methods.

### 3.2.3 Liskov Substitution Principle (LSP)

All subclasses of User (Patient, Doctor, Admin) can be used wherever a User reference is expected without breaking correctness. The UserService operates on User references; the UserFactory returns the correct subtype transparently. Similarly, any concrete PaymentStrategy can be substituted for another at runtime in PaymentContext without altering the payment processing flow. The decorator chain in AppointmentProcessor ensures each decorator is a valid substitute for the base AppointmentProcessor interface.

### 3.2.4 Dependency Inversion Principle (DIP)

High-level modules depend on abstractions, not concrete implementations. AppointmentServiceImpl depends on the AppointmentObserver interface (not InAppNotifier directly); Spring injects all registered observers as a List<AppointmentObserver>. Controllers depend on service interfaces (AppointmentService, PaymentService) rather than their impl classes. PaymentContext depends on PaymentStrategy interface, not on CardPaymentStrategy or UpiPaymentStrategy. This is achieved through Spring's dependency injection container throughout the project.

## 3.3 Design Patterns

### 3.3.1 Factory Method Pattern (Creational)

Location: com.appointment.pattern.factory.UserFactory

The Factory Method pattern centralizes the creation of User subtype objects (Patient, Doctor, Admin). The createUser(UserRole role, ...) method acts as the factory, returning the appropriate concrete subtype based on the role parameter. This decouples the client code (UserServiceImpl) from the constructors of specific user types.

Without this pattern, every place that creates a user would need to know the constructor details of each user type. Adding a new role (e.g., RECEPTIONIST) only requires adding a new case in the factory, with zero changes to any other class. The UserServiceImpl.register() method delegates entirely to UserFactory, keeping it free of creation logic.

### 3.3.2 Decorator Pattern (Structural)

Location: com.appointment.pattern.decorator

The Decorator pattern is applied to the appointment processing pipeline. The AppointmentProcessor interface defines the process() contract. BasicAppointmentProcessor provides the core logic (setting status to PENDING). ValidationAppointmentDecorator wraps it and adds pre-processing validation checks (null checks, slot availability). LoggingAppointmentDecorator wraps the entire chain and adds audit logging before and after processing.

In AppointmentServiceImpl.book(), the chain is assembled as: LoggingAppointmentDecorator(ValidationAppointmentDecorator(BasicAppointmentProcessor())).

This allows behaviours (validation, logging, future: authorization) to be composed dynamically without subclassing. New decorators can be added to the chain without changing existing code.

### 3.3.3 Observer Pattern (Behavioral)

Location: com.appointment.pattern.observer

The Observer pattern is used to decouple appointment status changes from their side effects (notifications). The AppointmentObserver interface defines onAppointmentStatusChanged(Appointment, AppointmentStatus). InAppNotifier is the concrete observer that persists Notification records and simulates email/SMS dispatch. AppointmentServiceImpl holds a List<AppointmentObserver> injected by Spring and calls notifyObservers() after every status-changing operation (book, cancel, reschedule, updateStatus, complete).

This ensures that adding a new notification channel (e.g., PushNotificationNotifier) requires only creating a new Spring component implementing AppointmentObserver — no changes to AppointmentServiceImpl or any other existing class.

### 3.3.4 Strategy Pattern (Behavioral)

Location: com.appointment.pattern.strategy

The Strategy pattern is applied to payment processing to allow runtime selection of the payment algorithm. The PaymentStrategy interface defines processPayment(Payment) and getMethodName(). Three concrete strategies are implemented: CardPaymentStrategy, UpiPaymentStrategy, and CashPaymentStrategy, each registered as a Spring bean. PaymentContext holds all strategies and selects the correct one by matching getMethodName() to the user's selection.

When a patient chooses a payment method on the UI, PaymentServiceImpl calls paymentContext.executePayment(payment, method), which dispatches to the correct strategy without any if-else or switch logic in the service. Adding a new method (e.g., NetBanking) only requires a new class implementing PaymentStrategy.

# 4. GitHub Repository

The complete source code is publicly available at:

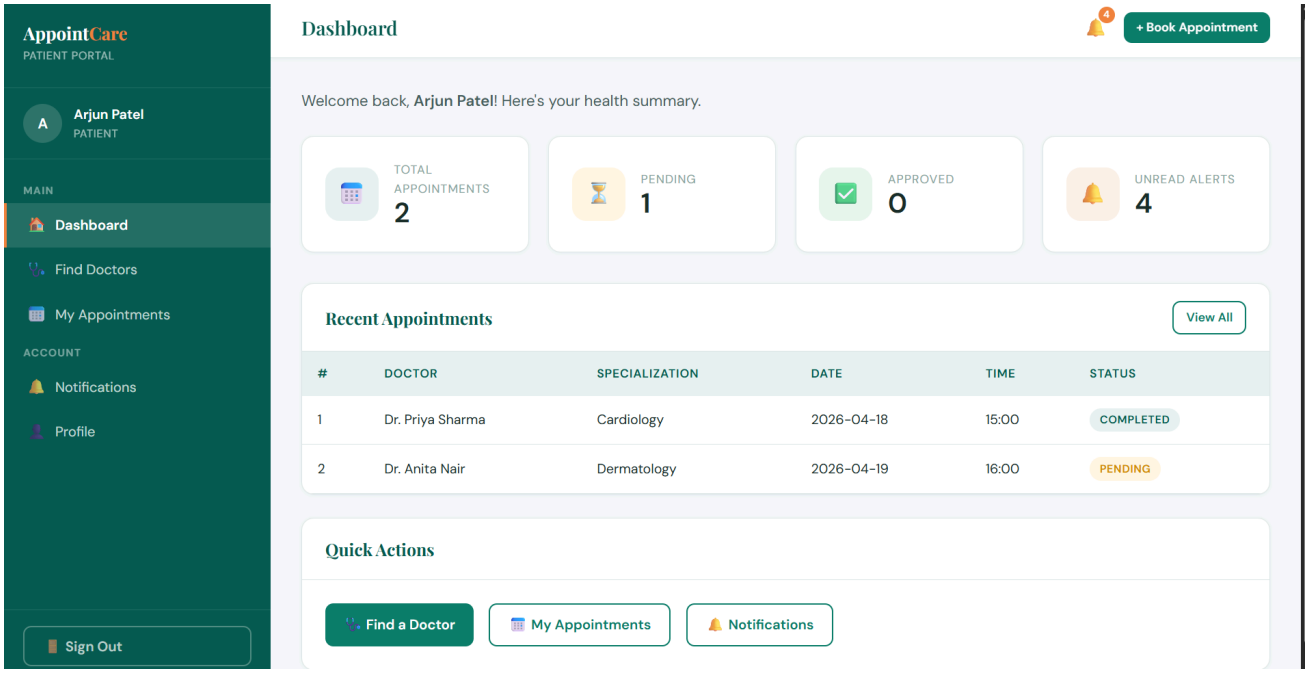
https://github.com/rtk5/AppointCare

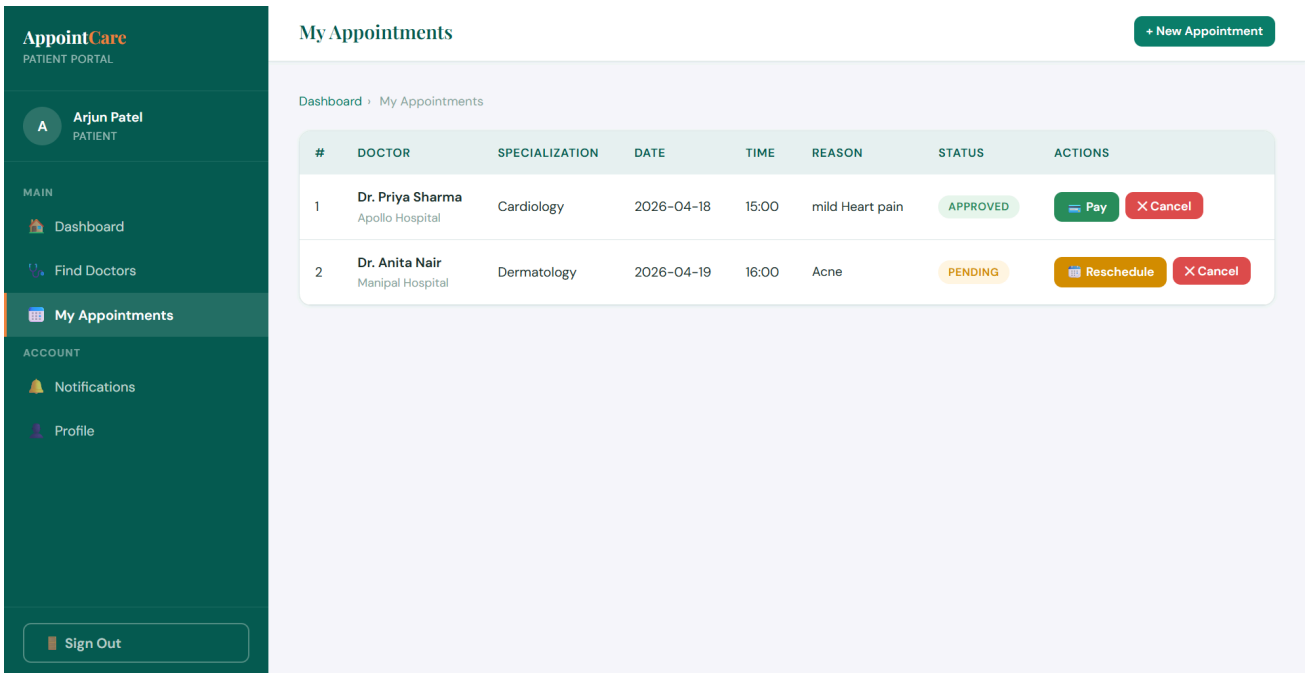
The repository is public and accessible. The main branch contains the complete Spring Boot application with all source files, Thymeleaf templates, and the Maven pom.xml build configuration.

# 5. Screenshots

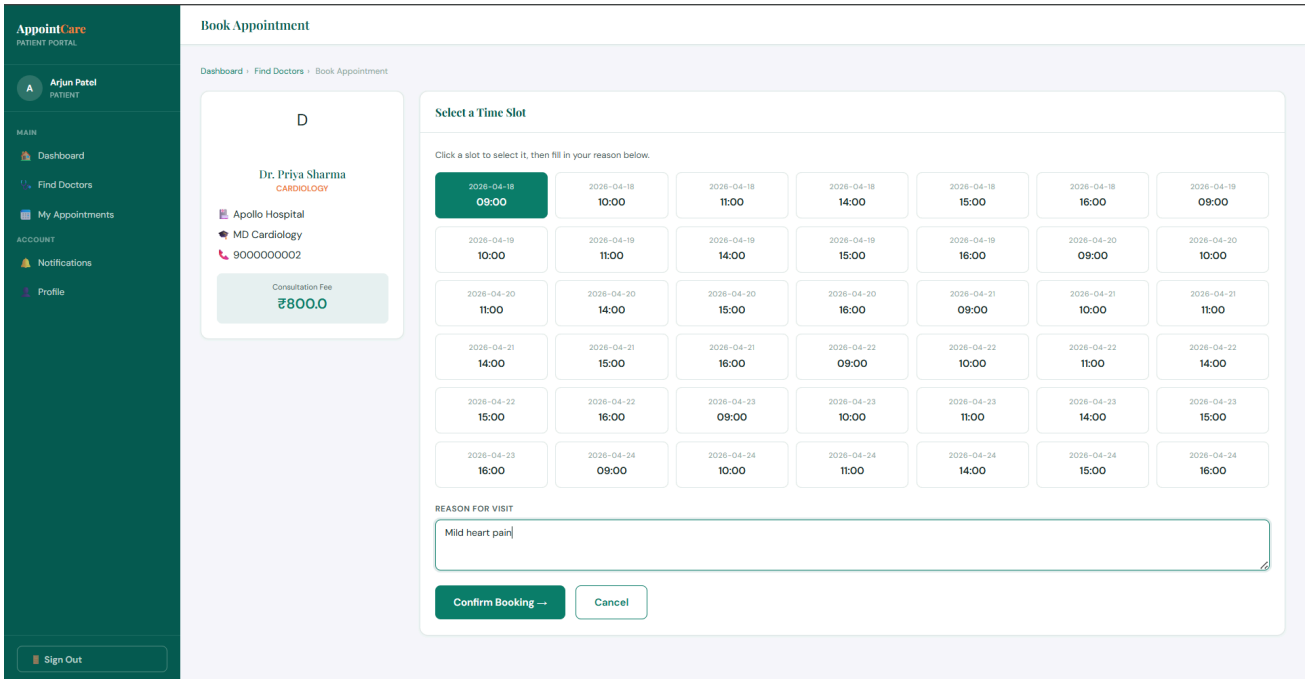
All screenshots below are captured with white backgrounds showing input values and output results.

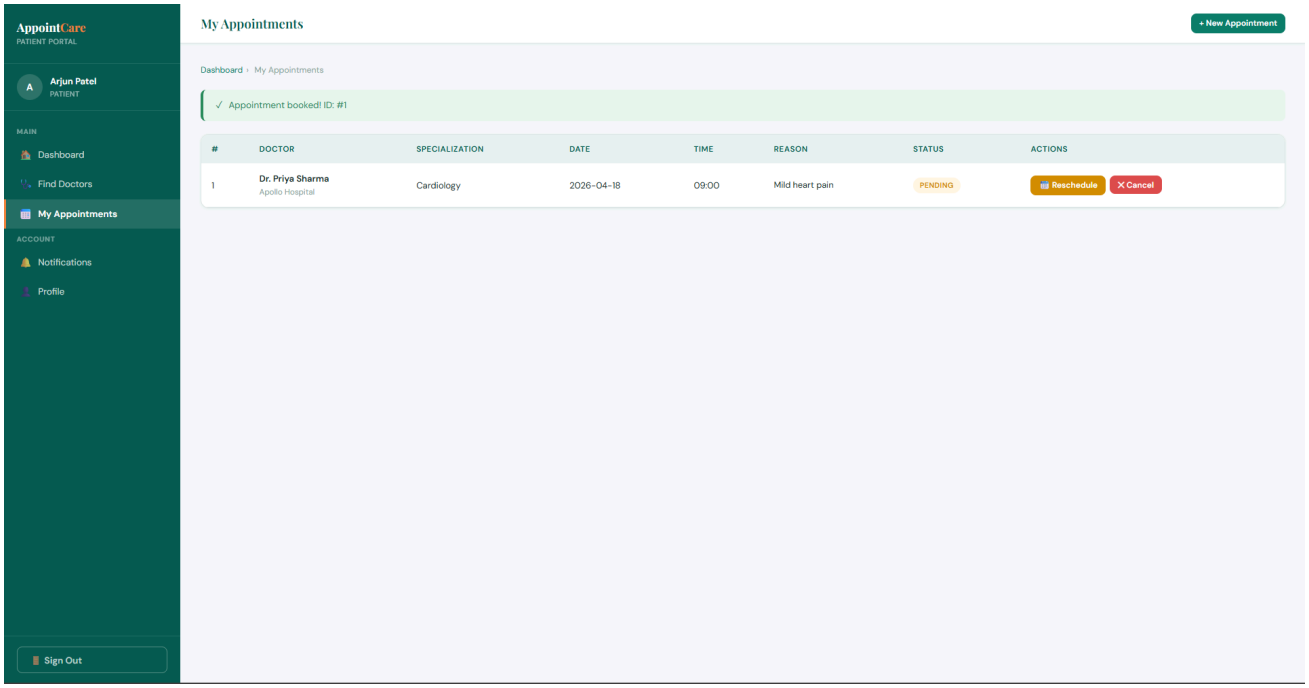
## 5.1 Patient Dashboard



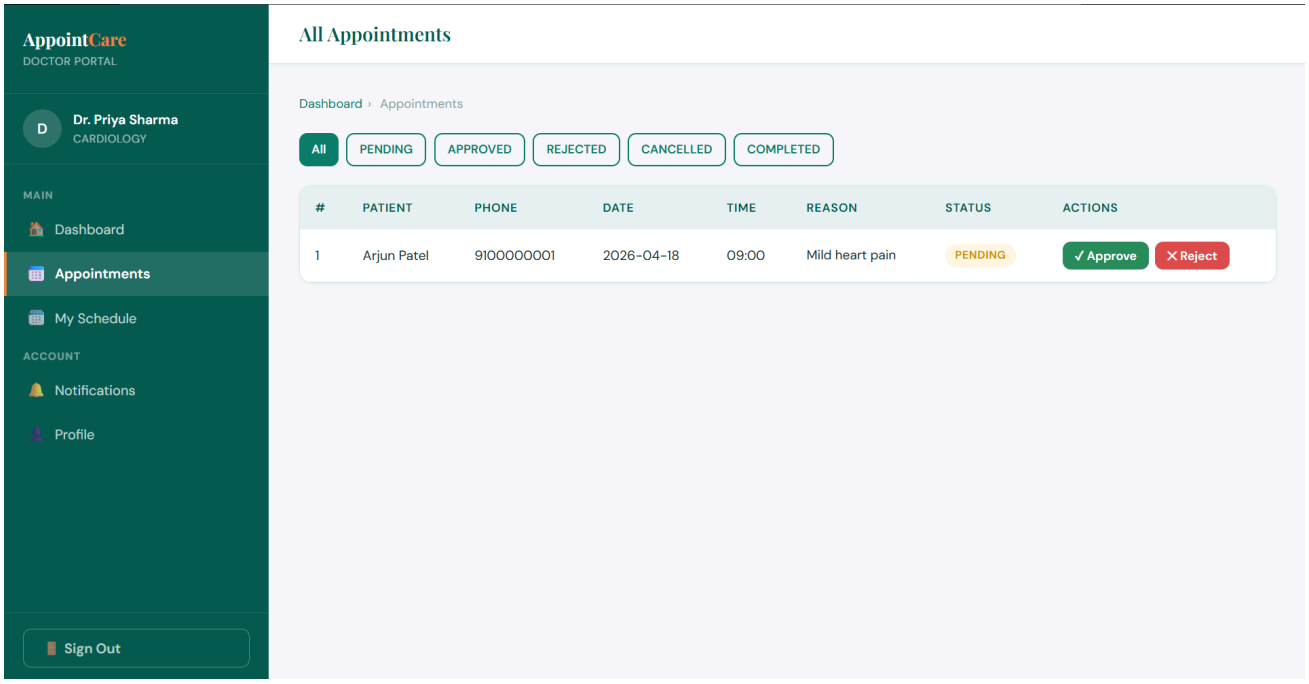



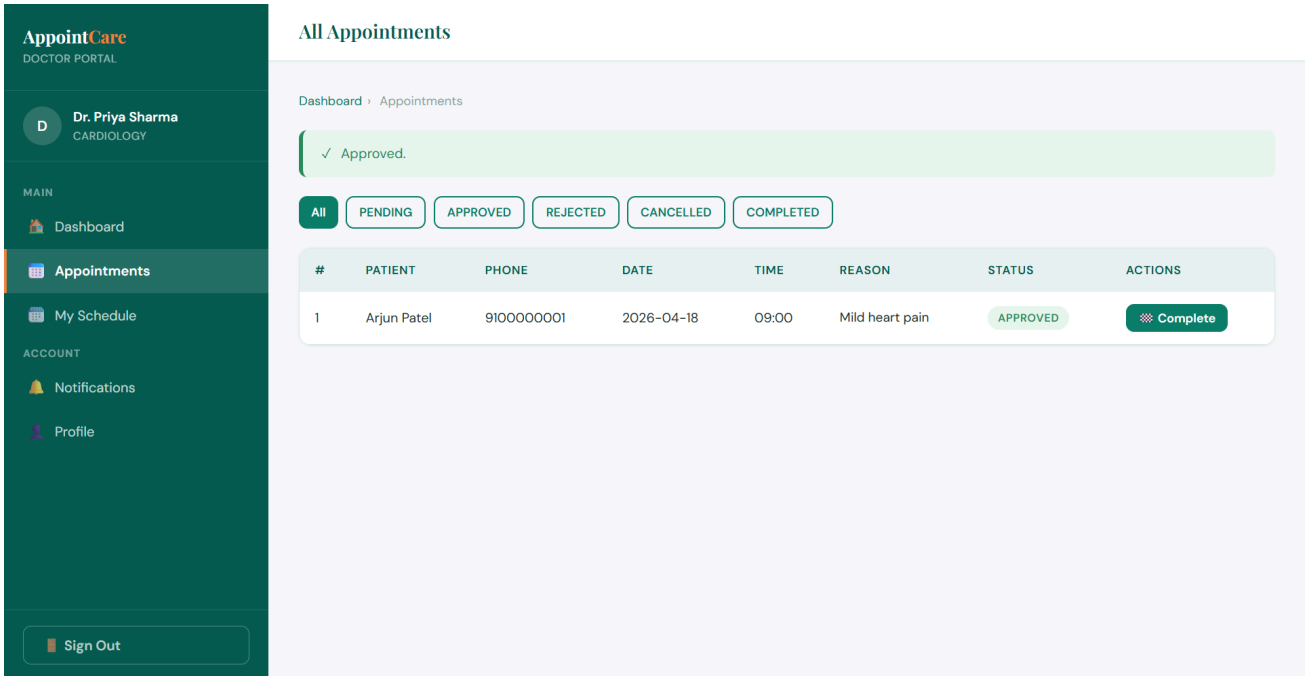
# 5.2 Book Appointment Form

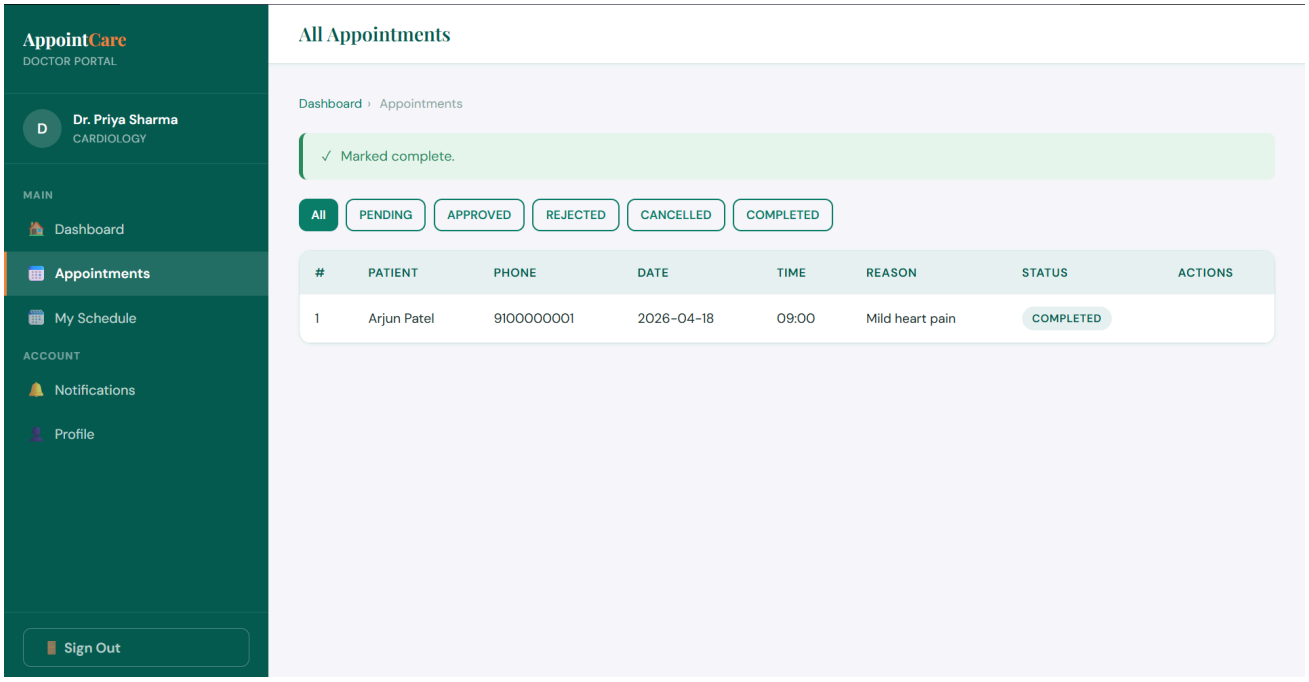


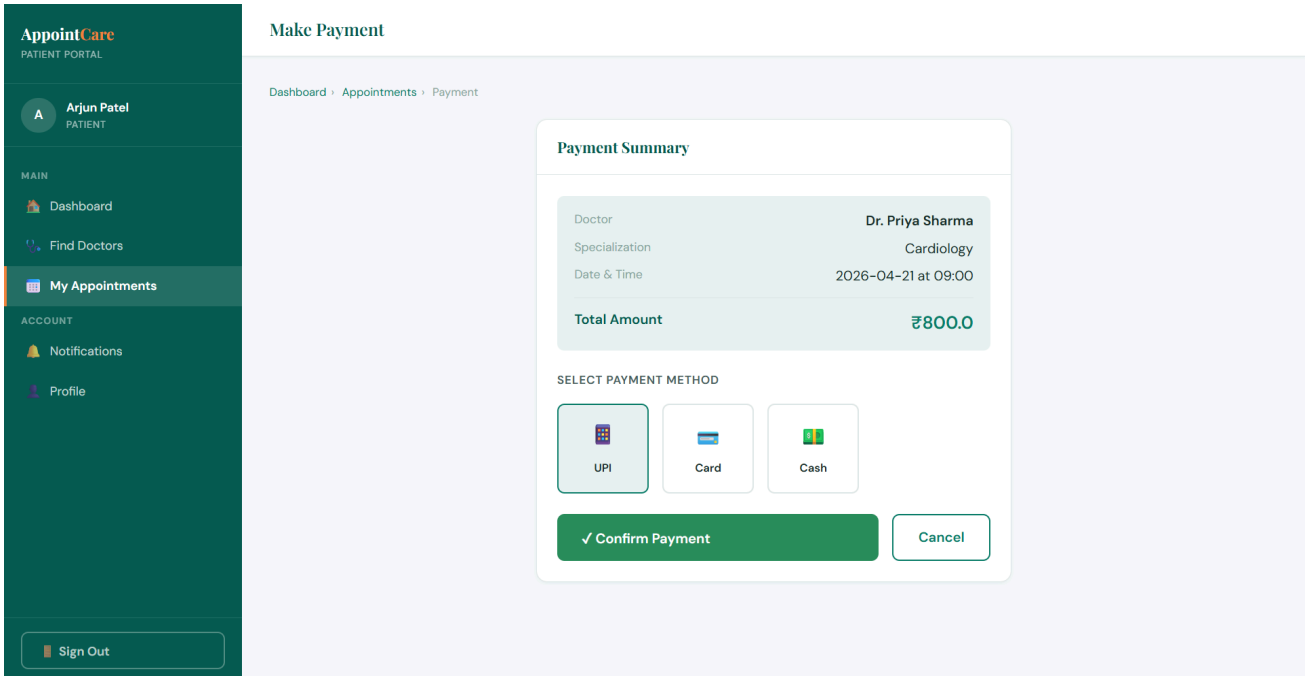
# 5.3 Doctor Dashboard & Appointment Approval



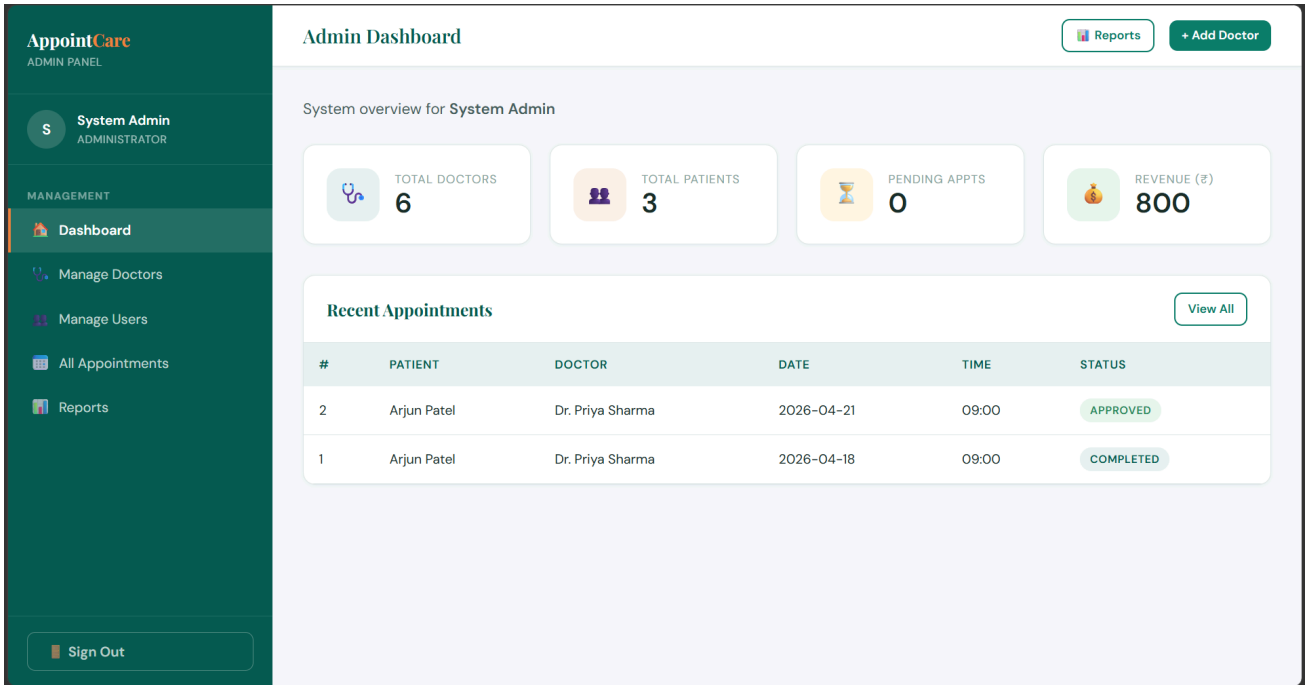



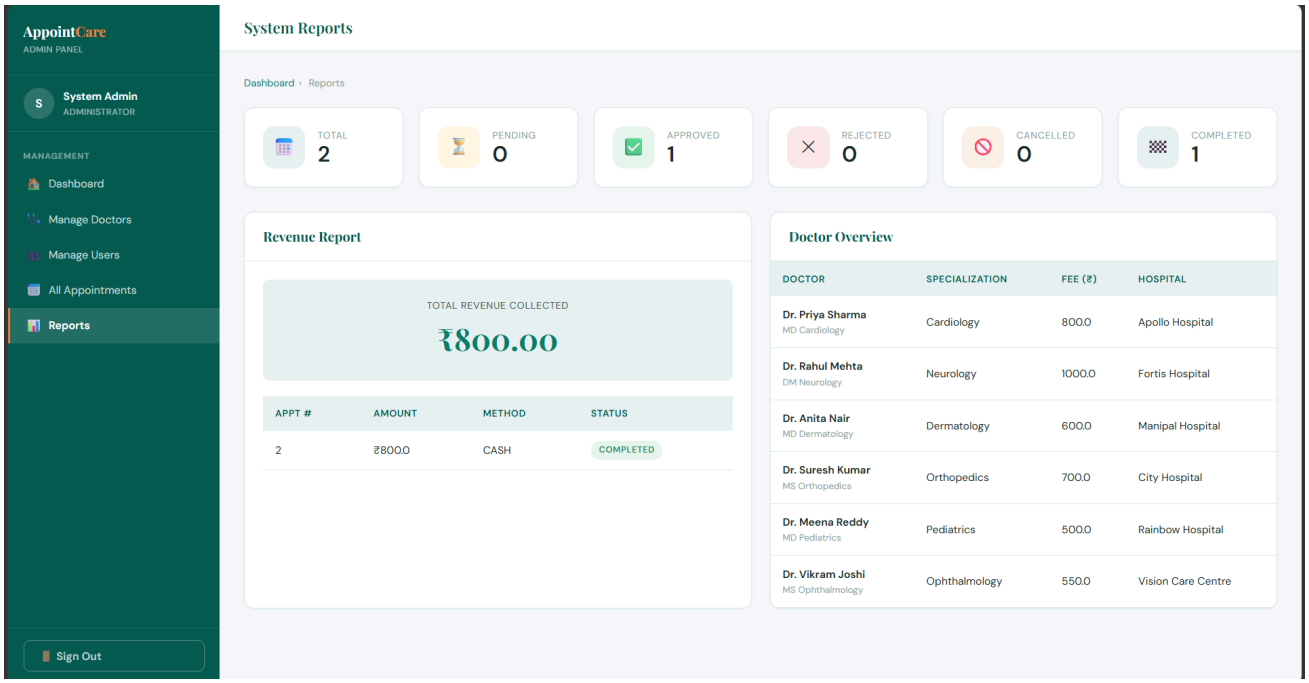


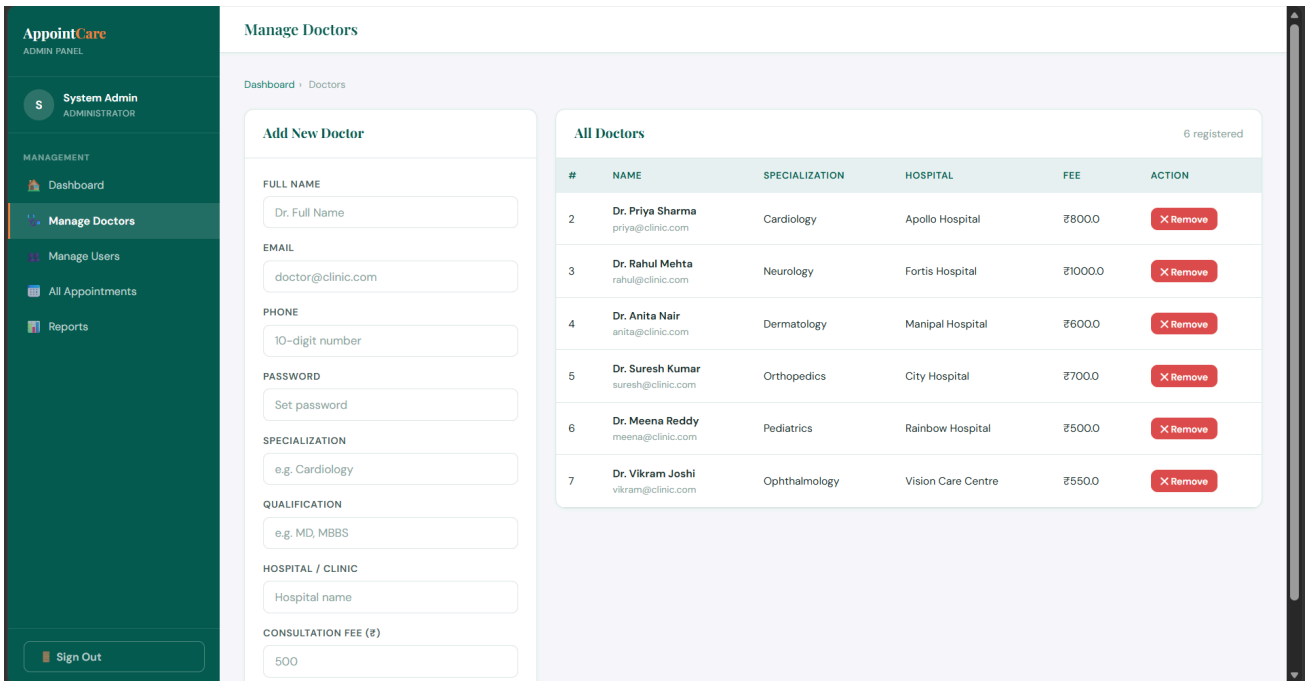

## 5.4 Payment Screen

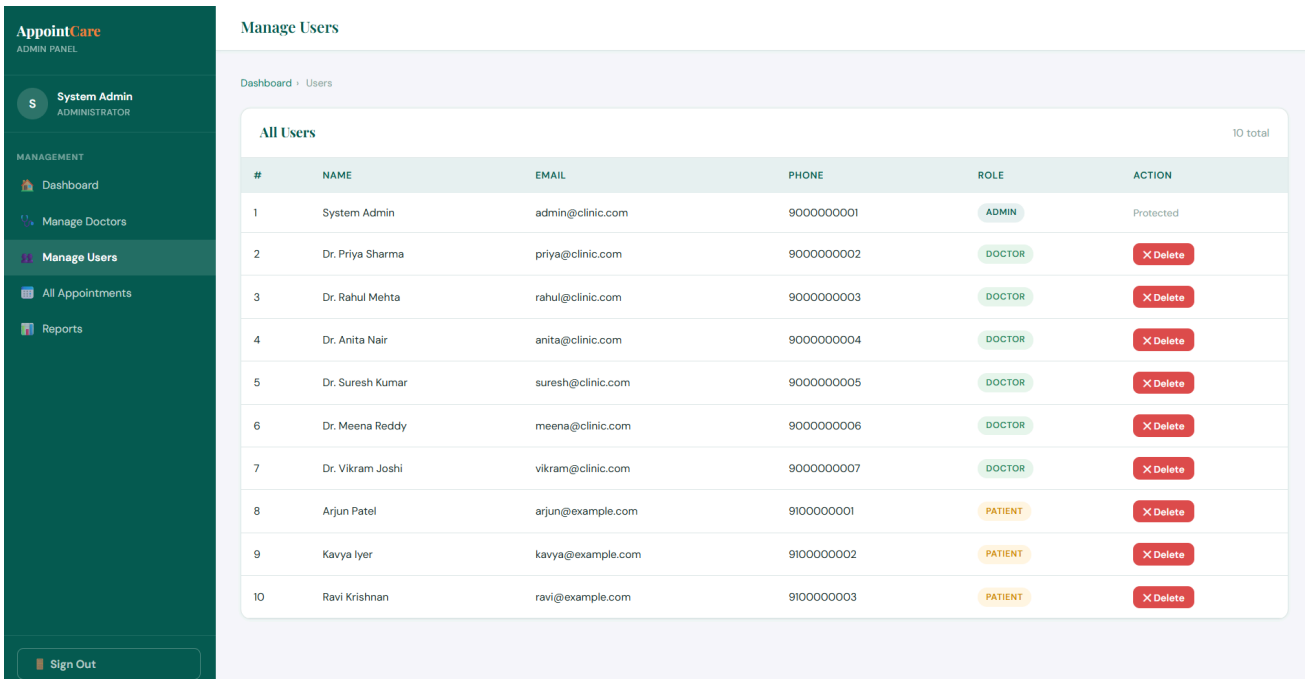


# 5.5 Admin Dashboard & Reports

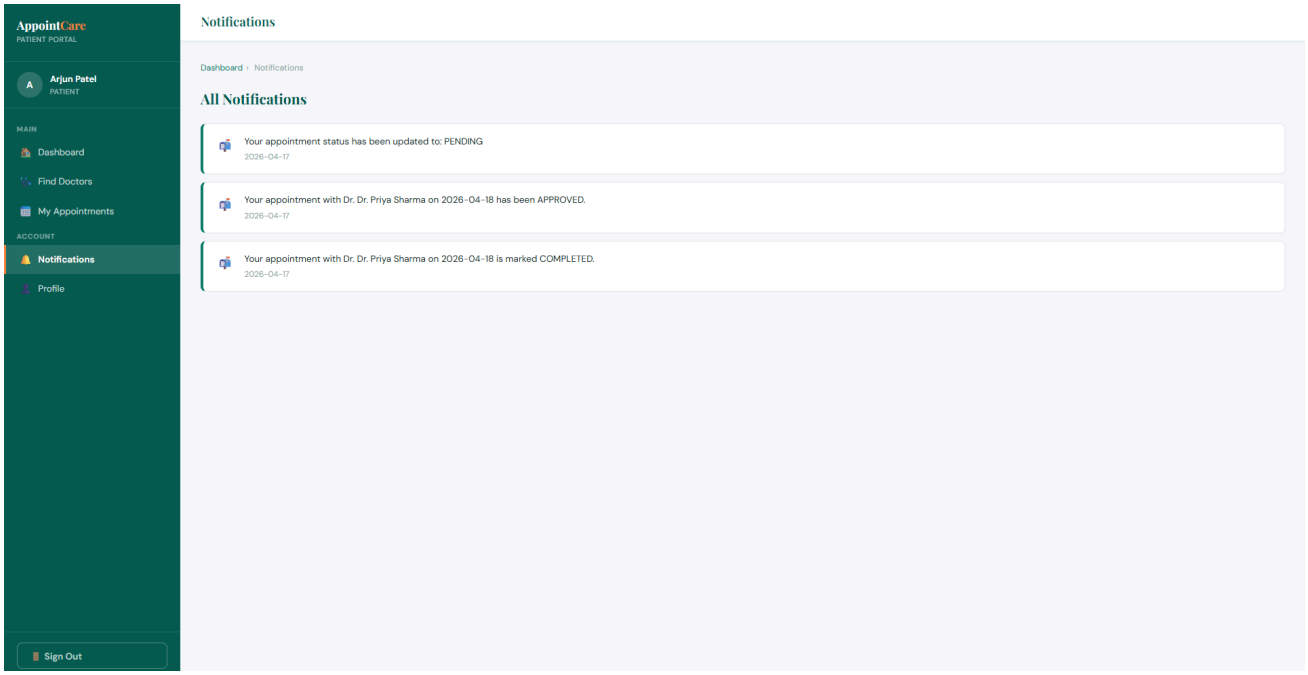



AppointCare
ADMIN PANEL
System Admin
ADMINISTRATOR
MANAGEMENT
Dashboard
Manage Doctors
Manage Users
All Appointments
Reports
Sign Out
Manage Doctors
Dashboard › Doctors
Add New Doctor
FULL NAME
Dr. Full Name
EMAIL
doctor@clinic.com
PHONE
10-digit number
PASSWORD
Set password
SPECIALIZATION
e.g. Cardiology
QUALIFICATION
e.g. MD, MBBS
HOSPITAL / CLINIC
Hospital name
CONSULTATION FEE (₹)
500
All Doctors
6 registered
# NAME SPECIALIZATION HOSPITAL FEE ACTION
2 Dr. Priya Sharma priya@clinic.com Cardiology Apollo Hospital ₹800.0 Remove
3 Dr. Rahul Mehta rahul@clinic.com Neurology Fortis Hospital ₹1000.0 Remove
4 Dr. Anita Nair anita@clinic.com Dermatology Manipal Hospital ₹600.0 Remove
5 Dr. Suresh Kumar suresh@clinic.com Orthopedics City Hospital ₹700.0 Remove
6 Dr. Meena Reddy meena@clinic.com Pediatrics Rainbow Hospital ₹500.0 Remove
7 Dr. Vikram Joshi vikram@clinic.com Ophthalmology Vision Care Centre ₹550.0 Remove


AppointCare
ADMIN PANEL
System Admin
ADMINISTRATOR
MANAGEMENT
Dashboard
Manage Doctors
Manage Users
All Appointments
Reports
Sign Out
Manage Users
Dashboard › Users
All Users
10 total
# NAME EMAIL PHONE ROLE ACTION
1 System Admin admin@clinic.com 9000000001 ADMIN Protected
2 Dr. Priya Sharma priya@clinic.com 9000000002 DOCTOR Delete
3 Dr. Rahul Mehta rahul@clinic.com 9000000003 DOCTOR Delete
4 Dr. Anita Nair anita@clinic.com 9000000004 DOCTOR Delete
5 Dr. Suresh Kumar suresh@clinic.com 9000000005 DOCTOR Delete
6 Dr. Meena Reddy meena@clinic.com 9000000006 DOCTOR Delete
7 Dr. Vikram Joshi vikram@clinic.com 9000000007 DOCTOR Delete
8 Arjun Patel arjun@example.com 9100000001 PATIENT Delete
9 Kavya Iyer kavya@example.com 9100000002 PATIENT Delete
10 Ravi Krishnan ravi@example.com 9100000003 PATIENT Delete

# 5.6 Notification Center



# 6. Individual Contributions

Each team member owned and implemented one complete major use case end-to-end (from Controller through Service to Repository and View), along with at least one design pattern and one minor feature.

| Name | Module / Use Case Owned | Design Pattern | Design Principle |
|---|---|---|---|
| Rithvik Matta | Appointment Booking & Rescheduling (PatientController, AppointmentServiceImpl, booking views) | Decorator Pattern | SRP |
| Rishil Abhijit Jalisatgi | Doctor Schedule & Approval (DoctorController, ScheduleServiceImpl, doctor views) | Observer Pattern | OCP |
| Retesh G S | Payment Processing (PaymentServiceImpl, PaymentContext, payment views) | Strategy Pattern | DIP |
| Rishi A Sheth | Admin Dashboard, User Management & Reporting (AdminController, UserFactory, admin views) | Factory Method Pattern | LSP |